

Mock Exam Solution

By:

- Ali AlRashedi (moderator)
- Hussain Ali

Disclaimer

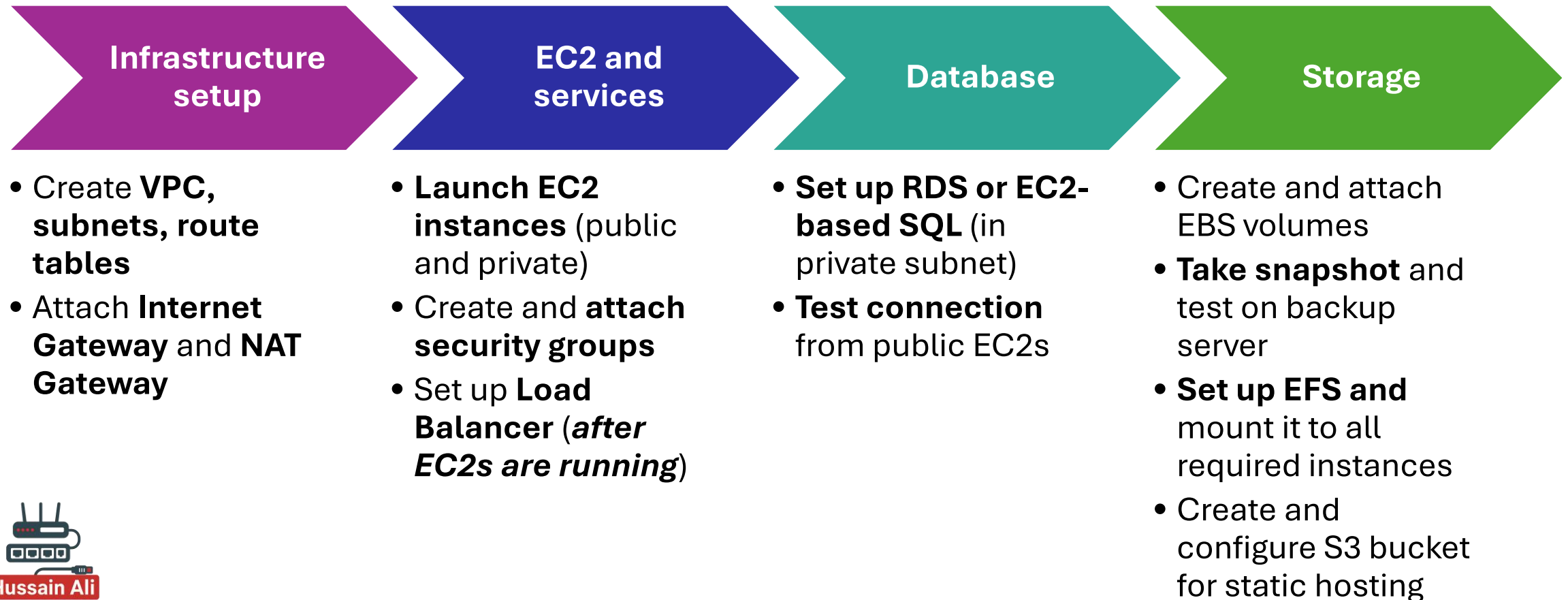
This mock review is for training only.

The exam will not repeat this exact setup.

But it covers all major concepts and helps you prepare well.

How to Approach Any Given Topology

Don't panic. Read the question carefully.



Task 1: VPC and Subnet Configuration

Task 1: VPC and Subnet Configuration

- Create a **VPC** named “**MockVPC**” with a **Class C, CIDR block**.
- Create **two public subnets** and **two private subnets** across **4 Availability Zones**.
- Attach an **Internet Gateway Called “Mockig”** to the VPC.
- Create a **NAT Gateway called “Mock-NAT”** in the public subnet to connect the private subnet.

AWS implementation: **Task 1: VPC and Subnet Configuration**

End of Part 1 – Infrastructure Setup (VPC, Subnets, Routing, IGW, NAT)

- Tips:
 - **Stick to given naming conventions** like MockVPC, MockIGW, etc., for clarity.
 - **Double-check subnet CIDRs and AZ distribution.** Each subnet should be in a different AZ.
 - **Don't forget to associate** the routing table with the **correct subnet**.
 - If your NAT Gateway isn't working, check the **Elastic IP and route table config**.
 - Have a clear **understanding of the public and private subnets**
 - Take a **screenshot** after each main step—VPC, subnet, IGW, NAT, routing.

Task 2: EC2 and Load Balancer Configuration

Task 2: EC2 and Load Balancer Configuration

- Launch **one Ubuntu EC2 instance** in each **public subnet**.
- Install a **Web Server** on each **public subnet**.
- Install **Windows Server 2022 Base** on **Win Server**.
- Install **Red Hat Linux** on **Red Server**.
- Launch an **Ubuntu EC2 instance** for the **Data server** and **Backup Server** in **private subnet 2**.
- Create **security groups (SG)**:
 - **Public subnet SG** should allow **SSH, HTTP/HTTPS, ICMP, MYSQL/Aurora, RDS, and NFS**.
 - **Private subnet SG** should only allow access from the public subnet SG (web servers) and **NFS**.
- Create an **Application Load Balancer** to balance traffic between the two Web Servers and demonstrate its working.
- Register the Web Servers in the ALB's target group and verify the setup.

AWS implementation:

Task 2: EC2 and Load Balancer Configuration

End of Part 2 – EC2 Instances & Load Balancer

- Tips:
 - Use **clear names for instances** (Ubuntu-Web-1, RedHat-1, etc.).
 - **Make sure your security groups allow only what's needed—don't overexpose ports.**
 - When setting up the **Load Balancer**:
 - **Add listeners correctly (HTTP on port 80).**
 - **Register only required instances.**
 - **Test from a browser** and screenshot it.
 - **If ALB isn't routing properly, check instance health checks and SG rules.**

Task 3: Backend and Storage Configuration

Task 3: Backend and Storage Configuration

- In private subnet 1:
 - Deploy **Amazon RDS** or **EC2-based DB** instance.
 - Set the master username to **admin** and the password to **master100**.
 - Configure availability zone to **no preference**.
- In private subnet 2:
 - In the **Data server**:
 - Use **EBS** volumes for storage (30 GiB **encrypted**).
 - Create a text file called **EBSmock.txt** inside the volume.
 - In Backup server:
 - Create a **snapshot** from the EBS volume and mount it on the **Backup server** and show the **EBSmock.txt**.
- Configure and verify shared file access using **Amazon EFS**
 - Create an **EFS file system** with mount targets in the required subnets.
 - Mount the **EFS file system** on:
 - **Both web servers** in the **public subnets**.
 - The **Backup server** in **private subnet 2**
- In **one instance**, create a text file named **EFSmock.txt** within the mounted EFS directory.
- Verify that the **EFSmock.txt** file is accessible from all instances connected to the **EFS file system**.

AWS implementation:

Task 3: Backend and Storage Configuration

End of Part 3 – Database Configuration (RDS, optional EC2 DB)

- **Tips:**
 - **Stick to provided credentials:** admin / master100.
 - **Choose "No Preference"** for AZ, unless instructed otherwise.
 - **Test RDS connection** from a public EC2 using MySQL command.

Task 4: S3 Bucket Configuration and Integration

Task 4: S3 Bucket Configuration and Integration

- Create an S3 Bucket
 - Enable **Access Control List (ACL)** for granular permission control.
 - Create an **index.html** file that includes **your name**.
 - Upload index.html to the S3 bucket and configure **Static website hosting**.

AWS implementation:

Task 4: S3 Bucket Configuration and Integration

End of Part 4 – EBS, EFS, and S3 Storage Setup

- **Tips:**

- **EBS volumes:** Always **mount and format** before use. Take a screenshot.
- **For snapshots,** show **both creation and restore steps.**
- **EFS:** Mount from at **least 2 instances** to show **shared access.**
- Always show **that the Text file is visible across AZs.**
- **S3 static hosting:** Use proper permissions and enable public read if needed.
- Upload an index.html with your name to prove it's your setup. **(if asked)**

Task 5: Validation and Screenshots

Task 5: Validation and Screenshots

- In the **Screenshots, Show that:**
 - Web servers can be accessed via the load balancer's DNS.
 - Instances in private subnets can access the internet through the NAT Gateway.
 - RDS is accessible from the EC2s in the public subnet.
 - EBS volume is attached and mounted correctly.
 - EFS is shared and accessible between public EC2s and the private subnet 2.
 - Public EC2 Instances should be able to ping each other.
 - Can access the static web page from the internet.
 - Can access the Ubuntu and Red Hat Servers from the CMD and Windows Server from the RDP.

AWS implementation: **Task 5: Validation and Screenshots**

End of Part 5 – Validation

- **Tips:**
 - Use the checklist from the mock (Page 5) and make sure each point is proven.
 - **Show CLI output for all instance checks.**
 - Don't skip showing access via ALB, NAT, EBS snapshot, and EFS sharing.
 - Keep screenshots labelled and in order.
 - **Use ping from the terminal to test networking.**

Task 6: Bonus Tasks

Note that this part is optional

Task 6: Bonus Tasks

- Create an **Application Load Balancer** to balance traffic between the **Red Hat server** and **web servers 2** and demonstrate its working.
(2 web servers + Red Hat in 1 load balancer)
- **Tip:** Use NGINX instead of httpd on Red Hat Linux
- Connect to **Windows Server** using **SSH**.
- **Note:** This task is for practice only. **It is very unlikely that similar** questions will appear in the actual exam.

AWS implementation: **Task 6: Bonus Tasks**

Note that this part is optional

End of Part 6 – Bonus Tasks

- **Tips:**
 - ***Bonus tasks aren't mandatory***, but they boost your confidence and show full understanding.
 - Test and show Red Hat + web servers in a new target group.
 - **For NGINX on Red Hat, show install, config file, and response in the browser.**
 - **SSH to Windows Server is tricky**—only try if you're confident.

Conclusion and general advice

Summary and General Tips

- Final Exam Tips:
 - Don't rely on console actions—use **CLI where required**.
 - Take clean, clear screenshots. (If required, write a note)
 - Know the purpose of each component: don't just click through.
 - ***If you're stuck, move on and come back—don't lose time.***
 - Please complete the mock exam from Moodle, and this mock exam before the actual exam day.

Thank you for watching
Wishing you the best

